

Reusabeer: Sistema de trazabilidad y gestión de envases retornables

Informe final del proyecto de aula

Joohan Nicolas Becerra González

Juan Sebastian Sanchez Triana

Universidad ECCI

Tecnología en Desarrollo Informático

Docente: Ing. Alied Victoria Garzón

Grupo 10

Resumen

Reusabeer es una aplicación web académica orientada a pequeñas y medianas empresas del sector de bebidas que necesitan dar seguimiento a envases retornables y a información operativa de inventario, producción y sostenibilidad. La solución integra una interfaz desarrollada en JavaScript, HTML y CSS con una API construida en Python mediante FastAPI y una base de datos PostgreSQL alojada en Supabase. En la versión revisada se identifican operaciones de envases, movimientos, lotes, vencimientos, producción, residuos, indicadores, usuarios y permisos, además de los componentes de autenticación, sesión y cierre de sesión. El seguimiento del proyecto se organizó en sprints. Para esta entrega se revisó el código, se verificó su sintaxis, se consolidaron capturas y se ajustaron los requerimientos, historias de usuario y diagramas. La revisión no sustituye las pruebas integrales con la base de datos del equipo. Se diferencia entre implementación observada en el código, capturas disponibles y pruebas funcionales aún por documentar.

Palabras clave: trazabilidad, FastAPI, PostgreSQL, Scrum, envases retornables.

1. Introducción y planteamiento del problema

En una empresa que distribuye bebidas en envases retornables, registrar manualmente entregas y devoluciones dificulta conocer qué recipientes continúan en circulación, quién registró sus movimientos y cuándo regresaron. A este problema se suman las necesidades de controlar los lotes de productos, sus vencimientos, la producción y las mermas. Reusabeer reúne esos procesos en una aplicación académica con información consultable por los diferentes usuarios del sistema.

La finalidad del desarrollo fue demostrar conocimientos de programación en Python mediante una aplicación conectada a una base de datos y una interfaz funcional. El proyecto evolucionó desde un enfoque inicial centrado en la trazabilidad del envase hacia un alcance que

incluye inventario, producción, residuos y reportes. La documentación conserva esta evolución para dar cuenta del trabajo realizado y no presentar como plan original todas las funciones agregadas durante los sprints.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación web para el registro y la consulta de envases retornables y su trazabilidad, integrada con inventario, producción, gestión de residuos y reportes, empleando Python, FastAPI y PostgreSQL.

2.2 Objetivos específicos

- Identificar los envases mediante un código y registrar los movimientos asociados a su circulación y retorno.
- Administrar productos, lotes, existencias y fechas de vencimiento en una base de datos persistente.
- Registrar procesos productivos, calcular indicadores de eficiencia y clasificar residuos.
- Consultar indicadores y reportes de sostenibilidad, con mecanismos de exportación compatibles con herramientas de oficina.
- Incorporar autenticación, manejo de sesiones, usuarios y datos de permisos por rol.
- Aplicar Scrum/Kanban y documentar requerimientos, casos de uso, clases y pruebas para sustentar el proyecto.

3. Alcance y versión final

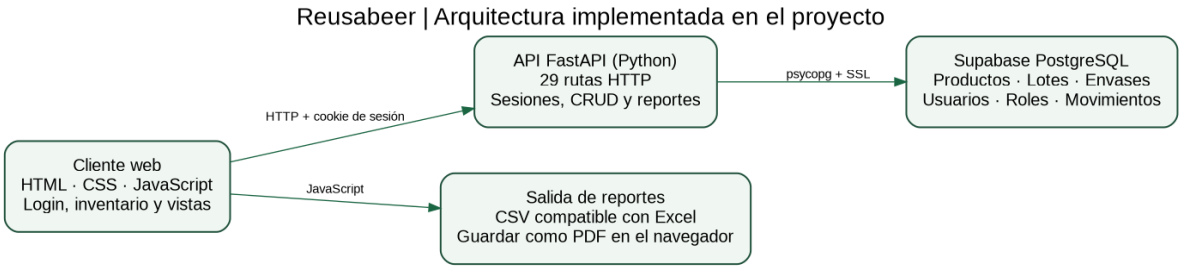
La versión examinada del repositorio incorpora los siguientes componentes: dashboard, inventario y lotes, productos vencidos y próximos a vencer, producción y mermas, destino de residuos, envases y trazabilidad, reportes, usuarios y roles. El frontend cuenta además con un formulario para iniciar sesión, la recuperación de la sesión vigente y una opción para cerrarla. Las acciones se apoyan en 29 rutas HTTP definidas en backend/main.py. El acceso a datos se configura mediante psycopg y variables de entorno. Algunas funciones de la base de datos, como el trigger que cambia el estado de un envase, no pueden certificarse solo con el ZIP porque no incluye el esquema SQL.

Se excluyen del alcance confirmado un sitio de producción públicamente desplegado, pruebas integrales automatizadas y garantías de autorización granular por rol en cada ruta. Estos aspectos no deben confundirse con el hecho de que las pantallas y sus operaciones principales estén implementadas en el código de desarrollo.

4. Arquitectura y herramientas

Figura 1

Arquitectura general identificada en el código



Nota. Diagrama elaborado a partir del frontend, backend y funciones de exportación incluidos en el ZIP.

La separación entre el navegador, la API y la base de datos facilita que las vistas consulten información persistente. FastAPI define rutas de consulta y modificación, modelos Pydantic para datos de entrada y middleware de autenticación. PostgreSQL almacena entidades operativas y

usuarios. La interfaz utiliza funciones JavaScript para consumir la API, construir las pantallas y generar archivos descargables.

Componente	Tecnología y uso
Backend	Python y FastAPI; servidor Uvicorn; modelos Pydantic.
Persistencia	PostgreSQL/Supabase mediante psycopg, con sslmode=require.
Interfaz	HTML, CSS y JavaScript en cliente web local.
Seguridad	bcrypt para contraseñas y firma HMAC-SHA256 para cookies de sesión.
Configuración	python-dotenv y archivo .env local no destinado a GitHub.
Trabajo colaborativo	Git, GitHub y prácticas Scrum/Kanban.
Documentación de pruebas	Swagger/OpenAPI, capturas de frontend, inspección sintáctica y matriz de pruebas.

5. Análisis de requerimientos

Las matrices actualizadas incluyen 39 requerimientos funcionales, junto con requisitos no funcionales de usabilidad, persistencia, integridad, seguridad y mantenimiento. Los requerimientos se clasifican por estado de respaldo: implementado en código, visible en captura, reportado por el equipo o sin confirmación suficiente. La matriz completa mantiene la codificación histórica RF1–RF34 y añade RF35–RF39 para consultas de sesión, cierre, protección de rutas y controles de seguridad.

Área	Funcionalidades principales
Envases y movimientos	Registrar/editar envases; registrar salida, entrega, devolución, reingreso y lavado; consultar historial por ID.
Inventario y vencimientos	Consultar productos, crear/editar/eliminar lotes, calcular alertas según fechas.
Producción y residuos	Registrar procesos, eficiencia y mermas, consultar destinos de residuos.
Dashboard y reportes	Mostrar indicadores operativos, reportes mensuales, CSV y función de impresión para PDF.
Usuarios y acceso	Crear usuarios, activar/desactivar, consultar roles y editar rol_permiso; login, session y logout.

Las funcionalidades propuestas originalmente para envases pendientes por ruta, medición automática de retrasos, novedades estructuradas y porcentaje de retorno no se encontraron como rutas independientes plenamente verificables. Se conservaron en la matriz con su condición real en lugar de declararlas terminadas sin evidencia.

6. Historias de usuario

Se conservaron las 18 historias de usuario identificadas en la documentación anterior y se agregaron HU-19 (restaurar sesión), HU-20 (cerrar sesión) y HU-21 (autorizar cada operación

por rol, considerada mejora de seguridad no acreditada en backend). Cada historia dispone de necesidad, beneficio, prioridad, criterios de aceptación y estado actualizado en el libro Excel adjunto.

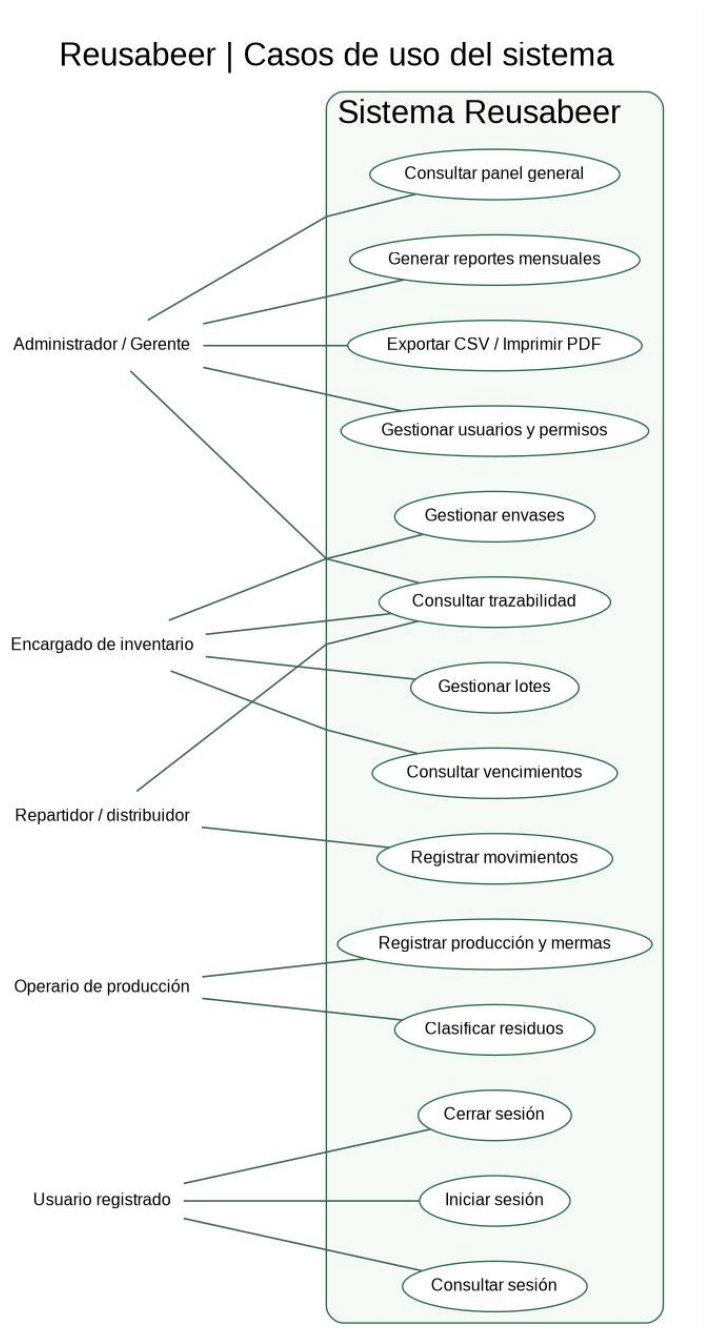
Historia	Descripción / verificación
HU-04	Consultar la trazabilidad por ID y el historial de movimientos; visible en captura E07.
HU-08–HU-09	Gestionar lotes y consultar vencimientos; visibles en E02–E04.
HU-10–HU-12	Registrar producción/mermas y residuos; visibles en E05–E06.
HU-13–HU-15	Consultar indicadores e informes, exportar CSV e imprimir a PDF; E01, E08 y código.
HU-16–HU-17	Administrar usuarios y la tabla rol_permiso; E09 y rutas de API.
HU-18–HU-20	Iniciar sesión, restaurarla y cerrarla; implementación identificada en código.
HU-21	Aplicar autorización por rol en cada operación; necesidad identificada, sin verificación de implementación.

7. Casos de uso

Los actores considerados son administrador/gerente, encargado de inventario, operario de producción, repartidor/distribuidor y usuario registrado. Los casos de uso se relacionan con la necesidad de consultar información, registrar operaciones y mantener la sesión del usuario. Las asociaciones del diagrama expresan funciones de negocio previstas para cada perfil, no una prueba de que todos los endpoints apliquen actualmente cada restricción por rol.

Figura 2

Casos de uso de Reusabeer



Nota. Modelo conceptual elaborado a partir del alcance y la revisión del código. No equivale a un resultado de pruebas de autorización.

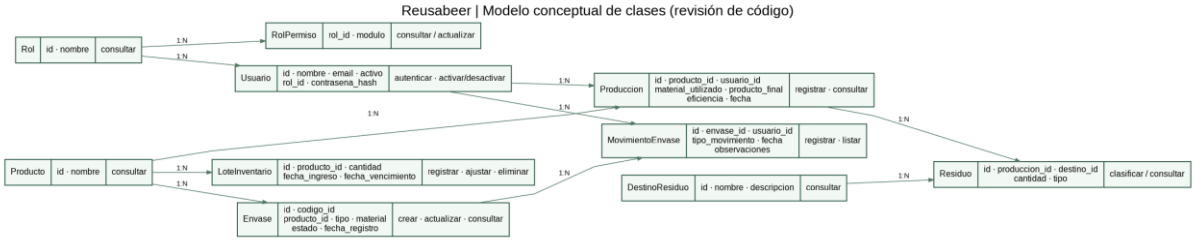
8. Diagrama de clases

El modelo conceptual representa las entidades principales de la información persistente: Usuario, Rol, RolPermiso, Producto, LoteInventario, Envase, MovimientoEnvase, Produccion, Residuo y DestinoResiduo. La relación RolPermiso se basa en el par rol_id y modulo que utiliza

el backend; no se modela una entidad Permiso separada sin fundamento en la implementación examinada.

Figura 3

Modelo conceptual de clases y relaciones



Nota. Diagrama orientativo derivado de consultas y atributos encontrados en backend/main.py; no sustituye el esquema SQL de Supabase.

9. Metodología ágil y seguimiento de sprints

Durante el proyecto se empleó un seguimiento por sprints y tablero de trabajo. Los documentos originales registran una etapa de definición y diseño, una revisión del alcance en el segundo corte, el diseño de persistencia, la conexión del backend y la integración de los módulos. Para evitar reconstruir fechas inexistentes se presentan los hitos comprobados por la documentación, sin inventar actas de reuniones ni ceremonias específicas de Scrum (Schwaber & Sutherland, 2020).

Etapa	Actividades / resultados
Sprint 1	Definición del caso, actores, primeros requerimientos, historias y modelos.
Sprint 2	Ajuste del foco de trazabilidad y reorganización documental.
Sprint 3	Diseño de tablas y datos iniciales en PostgreSQL.
Sprint 4	Conexión de FastAPI con PostgreSQL y pruebas iniciales de API.
Sprint 5	Migración/uso de Supabase e integración de envases, lotes, producción, residuos, reportes, usuarios.
Cierre técnico	Revisión del login y sus sesiones en código; actualización de documentación y enlace público de GitHub.

9.1 Dificultades, correcciones y retrospectiva

- El alcance inicial combinaba necesidades distintas; se reorganizó la propuesta y se documentó su ampliación real.

- La integración de base de datos y frontend requirió revisar consultas, estados de envases e historial de movimientos.
- Se sustituyeron datos simulados relevantes y se corrigieron errores de codificación; las capturas posteriores muestran registros operativos.
- La documentación de los sprints debía mantenerse sincronizada con el código: se actualizaron requerimientos, historias y matrices para evitar que siguieran describiendo una entrega sin login.

10. Pruebas y resultados sustentables

Se verificó la sintaxis del código Python mediante compileall y de los archivos JavaScript mediante node --check. Ambos procedimientos finalizaron sin errores en la versión del ZIP revisada el 10 de noviembre de 2026. También se identificaron 29 decoradores de rutas HTTP en el backend. Estas comprobaciones acreditan estructura sintáctica y disponibilidad de rutas, pero no certifican resultados funcionales ni la conexión privada a Supabase.

Clase de prueba	**Hallazgo comprobable / condición**
Estática: Python	Compilación de fuentes sin errores de sintaxis.
Estática: JavaScript	node --check sin errores en los scripts de interfaz.
Visual: pantallas	Nueve capturas E01–E09 reflejan datos en dashboard, inventario, vencimientos, producción, residuos, envases, reportes y usuarios.
Funcionales de API	Las rutas de CRUD y autenticación están presentes; su ejecución con datos reales no se realizó durante la revisión.
Casos negativos / seguridad	Pendientes de evidencia ejecutada: IDs duplicados, entradas inválidas, acceso por rol, bloqueo, cierre de sesión y persistencia.

La matriz de pruebas separa casos normales, límite y error con campos de procedimiento, resultado esperado y evidencia. Que un escenario figure como “diseñado” no significa que haya sido ejecutado exitosamente.

11. Inicio de sesión y seguridad

El frontend muestra un formulario de acceso y realiza una solicitud al endpoint POST /api/login. En el backend, autenticacion.py consulta el usuario en PostgreSQL y compara la

contraseña contra el hash bcrypt. Una vez autenticado, main.py emite una cookie de sesión HttpOnly, con atributo SameSite=Lax, firma HMAC-SHA256 y vencimiento de ocho horas. Las rutas GET /api/session y POST /api/logout permiten recuperar información de la sesión y cerrar el acceso. Un middleware devuelve HTTP 401 cuando una solicitud a la API no contiene una sesión válida (excepto el inicio de sesión).

La tabla rol_permiso relaciona roles con códigos de módulo y puede consultarse y editarse mediante endpoints específicos. Sin embargo, la revisión no encontró validación individual del rol en cada operación de backend. Por tanto, gestionar permisos no debe equipararse a garantizar autorización granular. En el entorno local la cookie usa Secure=False; para despliegue con HTTPS debe configurarse Secure=True. La lógica de bloqueo temporal por intentos fallidos también requiere una prueba y revisión de condición de fechas. Estas observaciones se indican como recomendaciones técnicas y no invalidan la existencia del flujo de inicio de sesión.

12. Resultados de las capturas y reportes

Las nueve capturas proporcionadas por el equipo muestran pantallas integradas antes de la última revisión del login. El panel general E01 presenta un indicador de eficiencia promedio, stock y cantidad de envases; E02 muestra lotes con ingreso, vencimiento y existencias; E03–E04 identifican lotes vencidos y vigentes; E05 refleja procesos y mermas; E06 organiza destinos de residuos; E07 muestra el estado y el historial de un envase; E08 contiene un reporte mensual con indicadores de sostenibilidad; E09 muestra usuarios, roles y estados de activación.

En la pantalla de reportes existe una opción denominada “Exportar Excel”, pero la implementación revisada descarga un archivo CSV con codificación UTF-8 que puede abrirse

con Excel. El botón PDF activa la impresión del navegador para que el usuario elija Guardar como PDF. La documentación identifica esos mecanismos de forma precisa.

13. Organización de la entrega y repositorio

El repositorio público del equipo es https://github.com/NicolasBece/reusabeer. Para facilitar la consulta de la profesora se preparó un README actualizado y una carpeta docs con informe y anexos APA 7, matrices, diagramas editables, capturas, presentación y guion del video. Al incorporarlos a la rama main, cualquier visitante puede leer los archivos públicos sin disponer de credenciales de edición. GitHub almacena el código y los documentos, pero no constituye por sí mismo un despliegue público del frontend y la API.

No deben compartirse variables de entorno reales, contraseñas ni cadenas de conexión. El ZIP recibido para revisión incluía el archivo backend/.env aunque Git no lo indicaba como versionado. Por esta razón, el paquete publicable añade reglas .gitignore y únicamente un archivo .env.example con valores ficticios.

14. Conclusiones

Reusabeer muestra la integración de una interfaz web con una API de Python y una base de datos PostgreSQL, y reúne funcionalidades útiles para trazabilidad de envases, inventario, producción y sostenibilidad. La incorporación del flujo de inicio y cierre de sesión constituye un avance frente a los documentos que lo identificaban como pendiente. La revisión del ZIP también permitió precisar diferencias entre permisos almacenados y autorización efectiva, así como describir correctamente la exportación de archivos.

En términos académicos, el proyecto permitió practicar el diseño de modelos, el consumo de API, consultas parametrizadas, control de versiones y metodología de trabajo colaborativo. La

versión documental actualizada refleja lo que se observa en el código y en las capturas. Para presentar resultados verificables en clase, el equipo debe ejecutar las pruebas funcionales de los escenarios más importantes y demostrar la aplicación en vivo.

Referencias

FastAPI. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com/

GitHub Docs. (s. f.). Adding a file to a repository. https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/

Python Software Foundation. (s. f.). Python documentation. https://docs.python.org/3/

Schwaber, K., & Sutherland, J. (2020). The Scrum guide. https://scrumguides.org/scrum-guide.html

Supabase. (s. f.). Supabase documentation. https://supabase.com/docs